

Experiment – 2

Name: Aryan

UID: 24BAI70179

Branch: CSE – AIML (CS221)

Section/Group: 24AIT-NTTP-2[A]

Semester: 5th

Date of Performance: /07/2026

Subject: Full Stack - II

Subject Code: 24CSP-337

EXPERIMENT – 2.1

1. AIM:

To design and implement a centralized state management system using Redux Toolkit for managing posts and platform-related data.

2. OBJECTIVE:

1. To understand the concept of centralized state management.
2. To implement Redux Toolkit for efficient state handling.
3. To create and configure a Redux Store.
4. To perform CRUD (Create, Read, Update, Delete) operations using Redux.
5. To connect React components with the Redux Store using React-Redux hooks.
6. To reduce prop drilling and improve application scalability.

3. SOFTWARE REQUIREMENTS:

1. Windows/Linux/macOS
2. Node.js (LTS Version)
3. NPM (Node Package Manager)
4. Visual Studio Code
5. Web Browser
6. React.js
7. React-Redux
8. Redux Toolkit

4. THEORY

Redux Toolkit is the official and recommended way to manage state in React applications. It provides a **centralized store** where application data is stored and managed. Instead of passing data from one component to another using props (prop drilling), Redux allows all components to access the same data from a single source called the **Redux Store**.

Redux Toolkit simplifies Redux development by reducing boilerplate code and providing utilities such as `createSlice()` and `configureStore()`. A slice contains the application's state, reducers, and actions in one place. Reducers update the state based on dispatched actions, while React components access the state using `useSelector()` and update it using `useDispatch()`.

In this experiment, posts are stored in the Redux Store, and CRUD operations such as Add, Read, Update, and Delete are performed efficiently. This centralized approach improves scalability, maintainability, and performance of React applications.

5. IMPLEMENTATION/PROCEDURE

1. Create a React application.
2. Install Redux Toolkit and React-Redux packages.
3. Configure the Redux Store using configureStore().
4. Create a Slice using createSlice().
5. Define the initial state and reducers.
6. Connect the Redux Store to the React application using Provider.
7. Access state using useSelector().
8. Update state using useDispatch().
9. Perform CRUD operations on posts.
10. Run the application and verify the output.

6. CODE

• Store.js

```
import { configureStore } from "@reduxjs/toolkit";
import postReducer from "../features/posts/postSlice";

export const store = configureStore({
  reducer: {
    posts: postReducer
  }
});
```

• Postslice.js

```
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
  posts: []
};

const postSlice = createSlice({
  name: "posts",
  initialState,
  reducers: {
    addPost: (state, action) => {
      state.posts.push(action.payload);
    },
    deletePost: (state, action) => {
      state.posts = state.posts.filter(
        post => post.id !== action.payload
      );
    }
  }
});

export const { addPost, deletePost } = postSlice.actions;
export default postSlice.reducer;
```

• App.js

```
import { useState } from "react";
import { useDispatch, useSelector } from "react-redux";
import { addPost, deletePost } from "../features/posts/postSlice";
```

```
function App() {
  const [text, setText] = useState("");
  const posts = useSelector(state => state.posts.posts);
  const dispatch = useDispatch();

  const handleAdd = () => {
    if (text === "") return;
    dispatch(addPost({
      id: Date.now(),
      text: text
    }));
    setText("");
  };

  return (
    <div>
      <h2>Redux Post Manager</h2>
      <input
        value={text}
        onChange={(e) => setText(e.target.value)}
        placeholder="Enter Post"
      />
      <button onClick={handleAdd}>Add</button>
      {posts.map(post => (
        <div key={post.id}>
          <p>{post.text}</p>
          <button onClick={() => dispatch(deletePost(post.id))}>
            Delete
          </button>
        </div>
      ))}
    </div>
  );
}

export default App;
```

7. OUTPUT

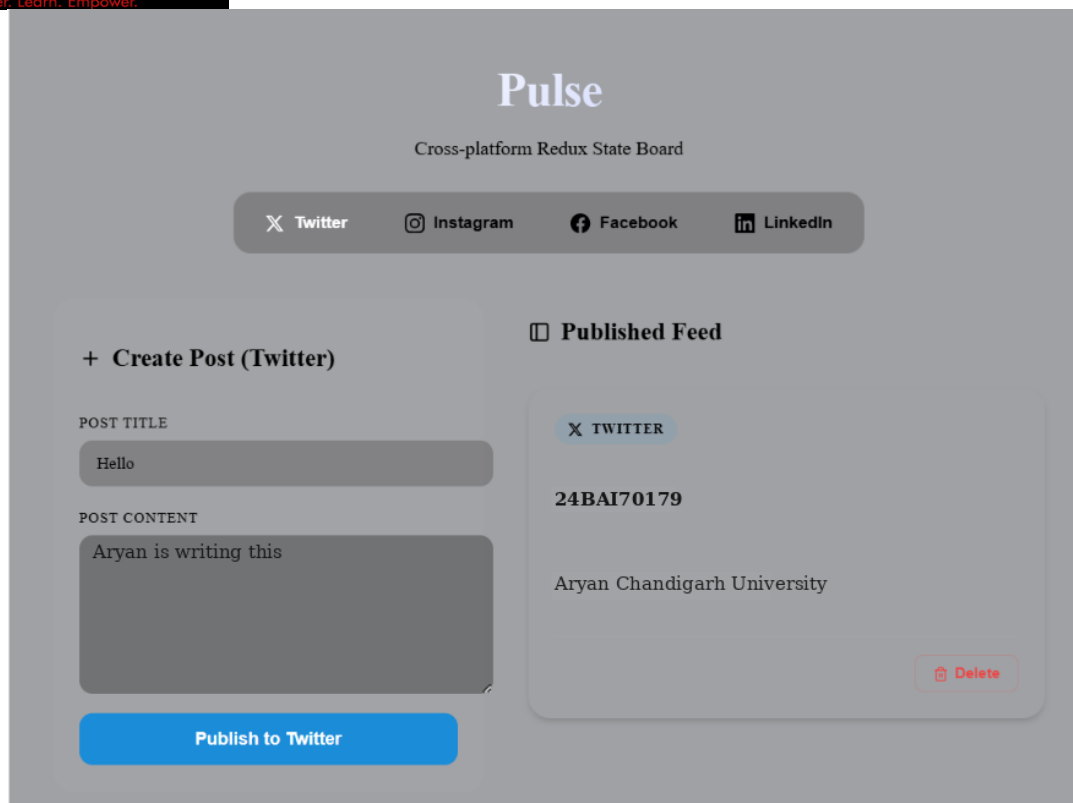


Fig 1.1: Pulse (Redux Toolkit Post Management System)

8. LEARNING OUTCOME

- Understand the concept of centralized state management.
- Understood centralized state management using Redux Toolkit.
- Learned to configure and manage the Redux Store.
- Implemented CRUD operations using Redux reducers.
- Connected React components with Redux using useSelector() and useDispatch().
- Improved application scalability, maintainability, and code organization.

EXPERIMENT – 2.2

1. AIM:

To optimize state access and improve application performance using memoized selectors and efficient rendering strategies.

2. OBJECTIVE:

- To understand the concept of derived state.
- To implement memoized selectors using Reselect.
- To reduce unnecessary component re-renders.
- To optimize state access in Redux applications.
- To improve performance in large-scale React applications.

3. SOFTWARE REQUIREMENTS:

1. Windows/Linux/macOS
2. Node.js (LTS Version)
3. NPM (Node Package Manager)
4. Visual Studio Code
5. Web Browser (Microsoft Edge)
6. React.js
7. React-Redux
8. Command Prompt / Terminal

4. THEORY

In React applications, frequent state updates can cause unnecessary component re-rendering, which affects application performance. Redux stores application state centrally, but repeatedly calculating derived data from the store can be inefficient.

Reselect is a library used with Redux to create memorized selectors. A memorized selector caches the result of a computation and recomputes it only when its input data changes. This avoids unnecessary calculations and improves rendering performance.

Derived state refers to data that is calculated from the existing Redux state rather than stored separately. By using selectors, components receive only the data they require, reducing unnecessary updates and improving scalability.

In this experiment, memoized selectors are implemented using Reselect to optimize state access, minimize re-renders, and enhance the overall performance of a React-Redux application.

5. IMPLEMENTATION/PROCEDURE

1. Create a new React application using Create React App.
2. Install Redux Toolkit, React-Redux, and Reselect.
3. Create a Redux Store using `configureStore()`.
4. Create a slice containing the application state.
5. Define basic selectors to access the Redux Store.
6. Create memoized selectors using `createSelector()`.

7. Compute derived state such as total posts and filtered posts.
8. Connect React components to the Redux Store using `useSelector()` and `useDispatch()`.
9. Use `React.memo()` and `useMemo()` wherever necessary to reduce unnecessary rendering.
10. Run the application using `npm start`.
11. Verify that selectors recompute values only when the state changes.

6. CODE

• Store.js

```
import { configureStore } from "@reduxjs/toolkit";
import postReducer from "../features/posts/postSlice";

export const store = configureStore({
  reducer: {
    posts: postReducer,
  },
});
```

• PostSlice.js

```
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
  posts: [
    { id: 1, text: "React Learning" },
    { id: 2, text: "Redux Toolkit" },
    { id: 3, text: "Memoized Selector" }
  ]
};

const postSlice = createSlice({
  name: "posts",
  initialState,
  reducers: {
    addPost: (state, action) => {
      state.posts.push(action.payload);
    },
    deletePost: (state, action) => {
      state.posts = state.posts.filter(
        post => post.id !== action.payload
      );
    }
  }
});

export const { addPost, deletePost } = postSlice.actions;
export default postSlice.reducer;
```

• PostSelector.js

```
import { createSelector } from "reselect";

const selectPosts = (state) => state.posts.posts;

export const selectAllPosts = createSelector(
  [selectPosts],
  (posts) => posts
);

export const selectPostCount = createSelector(
  [selectPosts],
```

```
(posts) => posts.length
);
```

• App.js

```
import React, { useState } from "react";
import "./App.css";

const PLATFORMS = [
  { id: "Twitter", label: "Twitter", limit: 280, accent: "twitter" },
  { id: "Instagram", label: "Instagram", limit: 2200, accent: "instagram" },
  { id: "Facebook", label: "Facebook", limit: 63206, accent: "facebook" },
  { id: "LinkedIn", label: "LinkedIn", limit: 3000, accent: "linkedin" },
];

function formatTime(date) {
  return date.toLocaleTimeString([], { hour: "numeric", minute: "2-digit" });
}

function App() {
  const [platform, setPlatform] = useState("Twitter");
  const [name, setName] = useState("");
  const [post, setPost] = useState("");
  const [queue, setQueue] = useState([]);

  const active = PLATFORMS.find((p) => p.id === platform);
  const remaining = active.limit - post.length;
  const overLimit = remaining < 0;

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!name.trim() || !post.trim() || overLimit) return;

    setQueue((prev) => [
      { id: Date.now(), platform, name: name.trim(), text: post.trim(), time: new Date() },
      ...prev,
    ]);
    setName("");
    setPost("");
  };

  return (
    <div className="desk">
      <div className="desk-inner">
        <header className="desk-header">
          <span className="eyebrow">Multi-channel composer</span>
          <h1>Transmission Desk</h1>
          <p className="subhead">Pick a channel, draft the post, watch it go live in the preview.</p>
        </header>

        <div className="composer-grid">
          <form className="composer-form" onSubmit={handleSubmit}>
            <label className="field">
              <span className="field-label">Name</span>
              <input
                type="text"
                placeholder="Who's posting?"
                value={name}
                onChange={(e) => setName(e.target.value)}
              />
            </label>
            <label className="field">
              <span className="field-label">Post</span>
              <textarea
                rows="6"
                placeholder={`Write your ${active.label} post...`}
              />
            </label>
          </form>
        </div>
      </div>
    </div>
  );
}
```

```

        value={post}
        onChange={(e) => setPost(e.target.value)}
      />
    </label>
    <div className={`char-meter ${overLimit ? "is-over" : ""}`}>
      {overLimit
        ? `${Math.abs(remaining)} characters over the ${active.label} limit`
        : `${remaining} characters left`}
    </div>
    <button
      type="submit"
      className={`btn-queue accent-${active.accent}`}
      disabled={!name.trim() || !post.trim() || overLimit}
    >
      Queue for {active.label}
    </button>
  </form>

  <section className="queue-section">
    <h2>Queued posts</h2>
    {queue.length === 0 ? (
      <p className="queue-empty">Nothing queued yet – add a post above to see it here.</p>
    ) : (
      <ul className="queue-list">
        {queue.map((item) => {
          const meta = PLATFORMS.find((p) => p.id === item.platform);
          return (
            <li key={item.id} className="queue-item">
              <span className={`queue-chip accent-${meta.accent}`}>{meta.label}</span>
              <div className="queue-content">
                <p className="queue-name">{item.name}</p>
                <p className="queue-text">{item.text}</p>
              </div>
              <span className="queue-time">{formatTime(item.time)}</span>
            </li>
          );
        })}
      </ul>
    )}
  </section>
</div>
</div>
</div>
);
}

export default App;

```

7. OUTPUT

The application successfully implemented memorized selectors using Reselect. State access was optimized, unnecessary component re-renders were reduced, and the overall performance of the React-Redux application was improved.

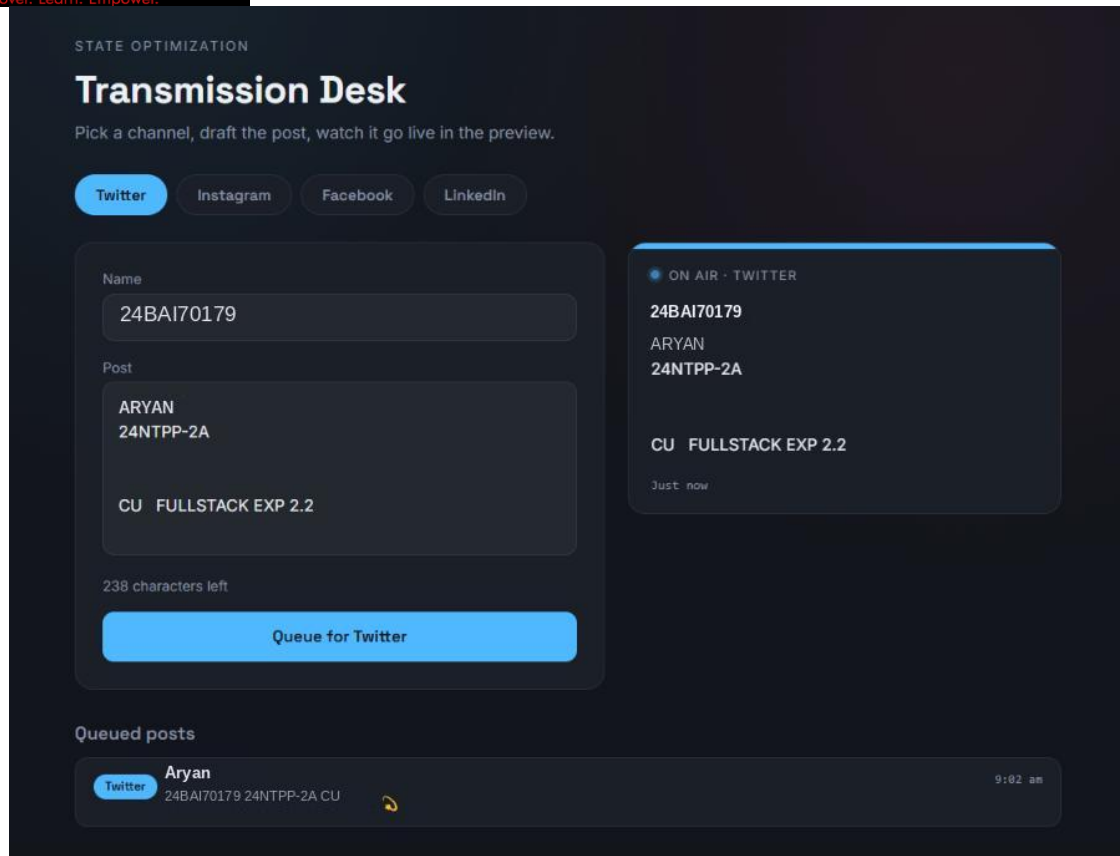


Fig 1.1: State Optimization with multi-channels

8. LEARNING OUTCOME

- Understood the concept of centralized state access using selectors.
- Learned the purpose of memoization in React applications.
- Implemented memoized selectors using createSelector().
- Computed derived state without duplicating data.
- Reduced unnecessary selector recomputations.
- Improved application rendering performance.
- Understood the use of React.memo() and useMemo().
- Connected React components with optimized Redux selectors.